Lab 1: Matrix Algebra and OLS by Hand

SOCIOL 723, Social Statistics II, Thursday, August 27

Wenhao Jiang

Table of contents

1 Goals

By the end of this lab you should be able to:

1. Organize an analysis in RStudio and Quarto so that it is reproducible.
2. Do the linear algebra of the linear model in `R`: transposes, products, inverses, rank, and traces.
3. Compute $\hat{\boldsymbol{\beta}} = (\boldsymbol{X}'\boldsymbol{X})^{-1}\boldsymbol{X}'\boldsymbol{y}$ by hand and verify that it matches `lm()`.
4. Construct the projection matrix $\boldsymbol{P}$ and the residual maker $\boldsymbol{M}$, and verify their properties numerically.
5. Verify the Frisch–Waugh–Lovell theorem and the omitted-variable-bias formula in real data.

Everything below runs on base `R` plus a handful of packages. Nothing here requires more linear algebra than we covered in lecture.

2 Getting set up

2.1 Packages

```
library(dplyr)      # data manipulation
library(ggplot2)    # graphics
library(broom)      # tidy() / augment() for model objects
library(modelsummary) # regression tables
```

If any of these are missing, install them once with `install.packages(c("dplyr", "ggplot2", "broom", "modelsummary"))`.

2.2 Working reproducibly

Three habits, which I will assume for every problem set:

- **Use an RStudio Project.** `File > New Project`. Never use `setwd()`; paths should be relative to the project root.
- **Write in Quarto (`.qmd`) or RMarkdown (`.Rmd`).** Your submission should be a rendered PDF produced from a source file that I could run myself.
- **Set a seed** before anything random: `set.seed(1)`.

A useful discipline: restart `R` (`Session > Restart R`, or `Cmd/Ctrl+Shift+F10`) and re-render from a clean session before you submit. If it does not run from scratch, it is not reproducible.

3 The data

We use an extract from the **General Social Survey** (NORC) cumulative file, pooling the 2010, 2012, 2014, 2016, 2018 and 2022 waves. The sample is restricted to respondents working full time, aged 25–64, with positive personal income and non-missing values on the analysis variables.

```
gss <- readRDS("../Data/gss_earnings.rds")
dim(gss)
#> [1] 3509   23
str(gss, give.attr = FALSE)
#> 'data.frame':    3509 obs. of  23 variables:
#>  $ year    : int  2010 2010 2010 2010 2010 2010 2010 2010 2010 2010 ...
#>  $ id      : int  1 13 20 21 29 35 38 42 44 50 ...
#>  $ lnearn  : num  10.66 9.87 10.66 10.26 10.46 ...
#>  $ realrinc: num  42735 19425 42735 28490 34965 ...
#>  $ educ    : num  16 12 16 12 16 15 16 18 17 12 ...
#>  $ degree  : Ord.factor w/ 5 levels "< HS"<"HS"<"Junior college"<..: 4 2 4 2 4 2 5 5 5 2 .
#>  $ ba      : int  1 0 1 0 1 0 1 1 1 0 ...
#>  $ exper   : num  9 40 13 31 11 36 31 9 2 22 ...
#>  $ age     : num  31 58 35 49 33 57 53 33 25 40 ...
#>  $ female  : int  0 0 0 0 1 0 0 0 0 1 ...
#>  $ black   : int  0 1 0 0 0 0 0 0 0 0 ...
#>  $ otherace: int  1 0 0 0 0 0 0 1 1 1 ...
#>  $ wordsum : num  6 5 6 6 8 7 3 8 3 2 ...
#>  $ pareduc : num  8 16 12 12 12 12 18 12 18 6 ...
#>  $ prestige: num  60 24 33 49 47 41 41 61 46 30 ...
#>  $ sei     : num  76.3 20.7 46.2 45.4 38.8 54.6 54.6 80.9 61.4 20.1 ...
#>  $ hours   : num  55 40 50 40 40 60 40 35 60 50 ...
#>  $ childs  : num  0 2 0 0 0 2 2 0 0 2 ...
#>  $ evermar : int  0 1 0 1 0 1 1 0 0 1 ...
#>  $ region  : Factor w/ 9 levels "New England",..: 2 2 2 2 2 2 2 2 2 1 ...
#>  $ stratum : int  2240 2242 2243 2243 2244 2245 2245 2246 2246 2247 ...
#>  $ psu     : int  22401 22421 22431 22431 22442 22451 22452 22461 22461 22471 ...
#>  $ wt      : num  1.127 0.64 0.447 0.554 0.39 ...
```

Variable	Meaning
`lnearn`	log of personal income, constant dollars (`realrinc`)
`educ`	years of schooling completed
`exper`	potential experience, max(age − educ − 6, 0)
`female`, `black`, `otherace`	indicator variables
`wordsum`	score on a 10-word vocabulary test (proxy for verbal ability)

Variable	Meaning
`pareduc`	highest education of either parent, in years
`prestige`, `sei`	occupational prestige and socioeconomic index
`hours`	usual hours worked last week
`region`	Census division (9 categories)
`stratum`, `psu`	GSS sampling stratum and primary sampling unit (used in Lab 2)
`wt`	post-stratification sampling weight

> **Note**
>
> The script that builds this file from the raw cumulative file is in `Data/build_gss_extract.R`. You do not need to run it.

A first look:

```
gss |>
  summarise(across(c(lnearn, educ, exper, wordsum, pareduc, hours),
                   list(mean = mean, sd = sd), .names = "{.col}_{.fn}")) |>
  t() |> round(2)
#>                [,1]
#> lnearn_mean   9.95
#> lnearn_sd     0.93
#> educ_mean    14.48
#> educ_sd       2.82
#> exper_mean   23.18
#> exper_sd     11.22
#> wordsum_mean  6.33
#> wordsum_sd    1.91
#> pareduc_mean 13.20
#> pareduc_sd    3.61
#> hours_mean   45.49
#> hours_sd     11.33
```

```
gss |>
  group_by(educ) |>
  summarise(m = mean(lnearn), n = n()) |>
  filter(n >= 20) |>
  ggplot(aes(educ, m)) +
  geom_point(aes(size = n), colour = "navy", alpha = .7) +
  geom_smooth(method = "lm", se = FALSE, colour = "firebrick", linewidth = .6) +
  labs(x = "Years of schooling", y = "Mean log earnings", size = "n") +
  theme_minimal(base_size = 10)
```

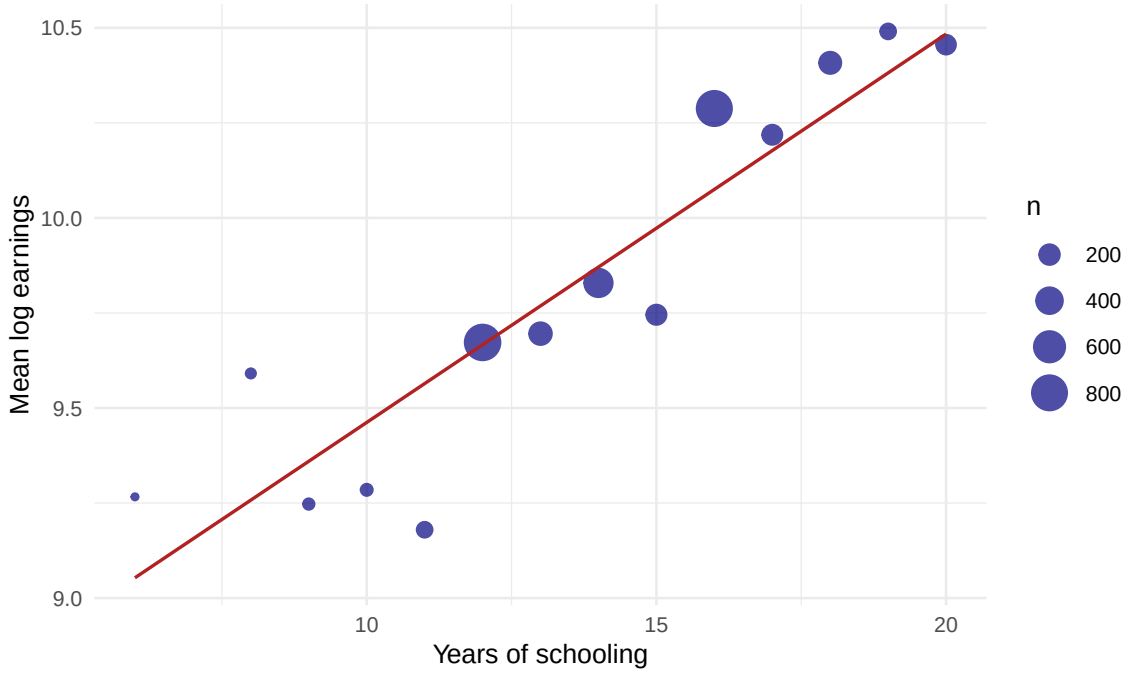

Figure 1: The CEF of log earnings given years of schooling, GSS 2010–2022.

The points are the **conditional expectation function** estimated cell by cell (this is what a *saturated* regression would fit). The line is the **best linear approximation** to it. Notice that the CEF is not exactly linear; it is convex above roughly 12 years, but the line summarizes it reasonably well.

4 Matrix algebra in R

4.1 Building blocks

```
A <- matrix(c(2, 1, 1, 3), nrow = 2)        # filled by column
b <- c(1, 2)

A
#>      [,1] [,2]
#> [1,]    2    1
#> [2,]    1    3
t(A)              # transpose
#>      [,1] [,2]
#> [1,]    2    1
#> [2,]    1    3
A %*% b           # matrix product (NOT A * b, which is elementwise)
#>      [,1]
#> [1,]    4
```

```
#> [2,]    7
solve(A)          # inverse
#>      [,1] [,2]
#> [1,]  0.6 -0.2
#> [2,] -0.2  0.4
solve(A) %*% A    # should be the identity
#>      [,1]                  [,2]
#> [1,]    1 -0.00000000000000005551
#> [2,]    0  1.00000000000000000000
diag(2)           # 2x2 identity
#>      [,1] [,2]
#> [1,]    1    0
#> [2,]    0    1
```

Two facts worth internalizing now, because they recur constantly:

```
crossprod(A)          # equals t(A) %*% A, but faster and more accurate
#>      [,1] [,2]
#> [1,]    5    5
#> [2,]    5   10
all.equal(crossprod(A), t(A) %*% A)
#> [1] TRUE

sum(diag(A))          # the trace
#> [1] 5
```

4.2 Rank, invertibility, and collinearity

$(\boldsymbol{X}'\boldsymbol{X})^{-1}$ exists if and only if $\boldsymbol{X}$ has full column rank. Here is what a rank failure looks like:

```
X_bad <- cbind(1, gss$educ, gss$educ * 2)   # third column is redundant
qr(X_bad)$rank                              # 2, not 3
#> [1] 2
try(solve(crossprod(X_bad)), silent = TRUE) |> class()
#> [1] "try-error"
```

`lm()` does not error, it drops the offending column and reports `NA`:

```
coef(lm(lnearn ~ educ + I(educ * 2), data = gss))
#> (Intercept)        educ I(educ * 2)
#>      8.2307      0.1187          NA
```

> **Warning**
>
> An `NA` coefficient is *never* a mystery: it means the column is an exact linear combination of the others. The most common cause is the dummy variable trap (including all J category indicators plus an intercept).

4.3 Positive definiteness

$\boldsymbol{X}'\boldsymbol{X}$ is positive semi-definite by construction, and positive definite when $\boldsymbol{X}$ has full column rank. We can check via eigenvalues:

```
X <- model.matrix(~ educ + exper + I(exper^2) + female + black, data = gss)
ev <- eigen(crossprod(X), only.values = TRUE)$values
range(ev)
#> [1]         68.09 2587232360.95
all(ev > 0)
#> [1] TRUE
```

The ratio of the largest to smallest eigenvalue is the **condition number**; a very large value warns of near-collinearity (numerically fragile, and substantively a warning that the data carry little independent information about some coefficient).

```
kappa(crossprod(X), exact = TRUE)
#> [1] 37994913
```

5 OLS by hand

5.1 The estimator

```
y <- gss$lnearn
X <- model.matrix(~ educ + exper + I(exper^2) + female + black, data = gss)

n <- nrow(X); k <- ncol(X)
c(n = n, k = k)
#>    n    k
#> 3509    6

XtX     <- crossprod(X)          # X'X   (k x k)
Xty     <- crossprod(X, y)       # X'y   (k x 1)
beta_hat <- solve(XtX, Xty)      # solve(A, b) is better than solve(A) %*% b
round(beta_hat, 4)
#>               [,1]
#> (Intercept)  7.3901
#> educ         0.1405
```

```
#> exper        0.0529
#> I(exper^2)  -0.0007
#> female      -0.3758
#> black       -0.1915
```

Compare with `lm()`:

```
fit <- lm(lnearn ~ educ + exper + I(exper^2) + female + black, data = gss)
all.equal(as.vector(beta_hat), unname(coef(fit)))
#> [1] TRUE
```

> **Tip**
>
> `solve(A, b)` solves the linear system directly instead of forming the inverse. It is faster and numerically more stable. In production code you would go further and use the QR decomposition, which is what `lm()` actually does: `qr.solve(X, y)`.

```
all.equal(as.vector(qr.solve(X, y)), as.vector(beta_hat))
#> [1] TRUE
```

5.2 Residuals, s^2, and standard errors

```
e   <- y - X %*% beta_hat           # residuals
s2  <- sum(e^2) / (n - k)           # unbiased estimator of sigma^2
V   <- s2 * solve(XtX)              # classical variance matrix
se  <- sqrt(diag(V))

round(cbind(estimate = beta_hat, se = se,
            t = beta_hat / se), 4)
#>                          se
#> (Intercept)  7.3901 0.0990  74.624
#> educ         0.1405 0.0051  27.537
#> exper        0.0529 0.0055   9.686
#> I(exper^2)  -0.0007 0.0001  -6.564
#> female      -0.3758 0.0279 -13.456
#> black       -0.1915 0.0386  -4.966
```

```
round(summary(fit)$coefficients, 4)
#>             Estimate Std. Error t value Pr(>|t|)
#> (Intercept)   7.3901     0.0990  74.624        0
#> educ          0.1405     0.0051  27.537        0
#> exper         0.0529     0.0055   9.686        0
#> I(exper^2)   -0.0007     0.0001  -6.564        0
#> female       -0.3758     0.0279 -13.456        0
```

```
#> black        -0.1915     0.0386  -4.966        0
```

Identical. The default standard errors that `R` prints are exactly $\sqrt{\text{diag}(s^2(\boldsymbol{X}'\boldsymbol{X})^{-1})}$, i.e. they *assume homoskedasticity*. Next week we replace them.

5.3 The mechanical orthogonality conditions

```
round(crossprod(X, e), 8)     # X'e = 0, up to floating point
#>                    [,1]
#> (Intercept)  0.00000000
#> educ         0.00000000
#> exper        0.00000000
#> I(exper^2)  -0.00000004
#> female       0.00000000
#> black        0.00000000
round(sum(e), 8)              # residuals sum to zero (constant included)
#> [1] 0
round(cor(X %*% beta_hat, e), 8)  # fitted values orthogonal to residuals
#>      [,1]
#> [1,]    0
```

These are identities, true in *every* sample no matter how badly specified the model is. They are not evidence of anything.

6 The geometry: $\boldsymbol{P}$ and $\boldsymbol{M}$

With $n = 3509$, the matrix $\boldsymbol{P}$ is 3509×3509, about 0.1 GB if we formed it. So we work with a random subsample to demonstrate the properties, and use the implicit form for the full data.

```
set.seed(1)
idx <- sample(n, 300)
Xs  <- X[idx, ]; ys <- y[idx]

P <- Xs %*% solve(crossprod(Xs)) %*% t(Xs)
M <- diag(nrow(Xs)) - P

## symmetric
all.equal(P, t(P))
#> [1] TRUE
## idempotent
all.equal(P %*% P, P)
#> [1] TRUE
all.equal(M %*% M, M)
#> [1] TRUE
```

```
## orthogonal to each other
max(abs(P %*% M))
#> [1] 0.0000000000000003427
## P X = X and M X = 0
all.equal(P %*% Xs, Xs, check.attributes = FALSE)
#> [1] TRUE
max(abs(M %*% Xs))
#> [1] 0.0000000000389
## traces are the dimensions of the two subspaces
c(trace_P = sum(diag(P)), k = ncol(Xs),
  trace_M = sum(diag(M)), n_minus_k = nrow(Xs) - ncol(Xs))
#>   trace_P         k   trace_M n_minus_k
#>         6         6       294       294
```

6.1 Leverage

The diagonal of $\boldsymbol{P}$ is the leverage h_{ii}. On the full sample we get it without ever forming $\boldsymbol{P}$:

```
h <- hatvalues(fit)
c(mean = mean(h), theoretical = k / n, max = max(h))
#>        mean theoretical         max
#>     0.00171     0.00171     0.01570
```

```
tibble::tibble(h = h, educ = gss$educ, exper = gss$exper) |>
  ggplot(aes(educ, h)) +
  geom_jitter(width = .18, alpha = .18, size = .5, colour = "navy") +
  geom_hline(yintercept = k / n, linetype = 2, colour = "firebrick") +
  labs(x = "Years of schooling", y = expression(h[ii])) +
  theme_minimal(base_size = 10)
```

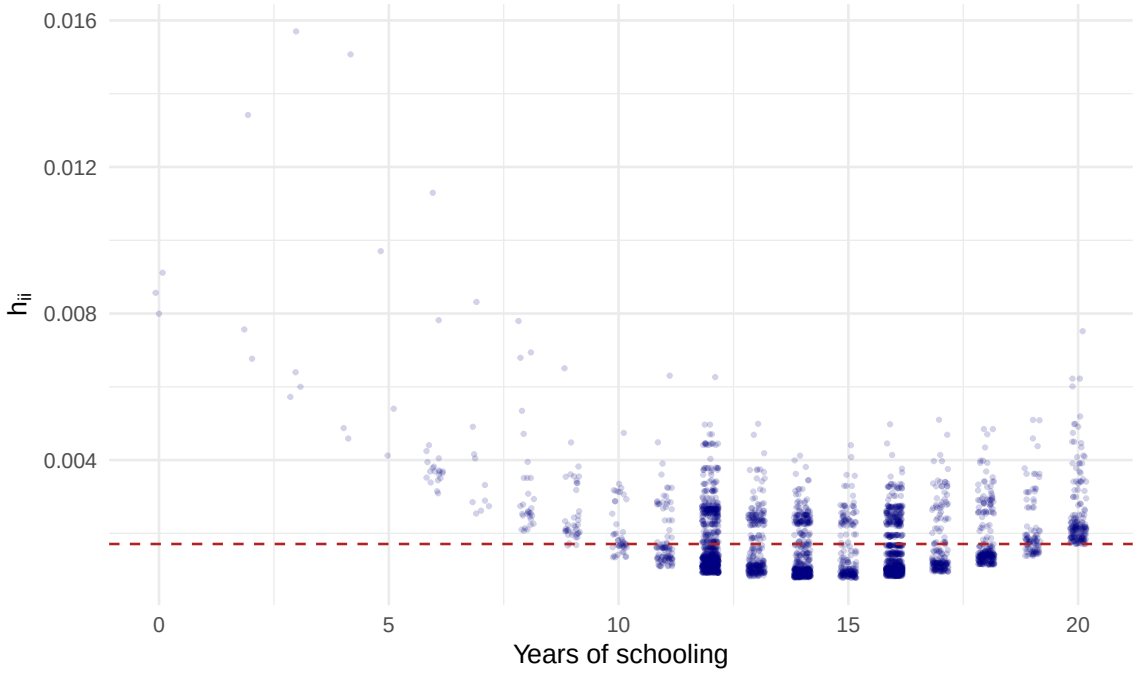

Figure 2: Leverage against years of schooling. The high-leverage observations are those with unusual *covariate* values, not unusual outcomes.

6.2 The sum-of-squares decomposition

```
yhat <- fitted(fit)
TSS <- sum((y - mean(y))^2)
ESS <- sum((yhat - mean(y))^2)
RSS <- sum(residuals(fit)^2)

c(TSS = TSS, ESS_plus_RSS = ESS + RSS)
#>          TSS ESS_plus_RSS
#>         3062         3062
c(R2_manual = ESS / TSS, R2_lm = summary(fit)$r.squared)
#> R2_manual     R2_lm
#>    0.2326    0.2326
```

And the model-comparison reading of R^2: it is PRE against the intercept-only model.

```
fit_c <- lm(lnearn ~ 1, data = gss)              # MODEL C
SSE_C <- sum(residuals(fit_c)^2)
SSE_A <- sum(residuals(fit)^2)                    # MODEL A
c(PRE = (SSE_C - SSE_A) / SSE_C, R2 = summary(fit)$r.squared)
#>    PRE     R2
#> 0.2326 0.2326
```

7 Frisch–Waugh–Lovell

We now verify FWL on the return to schooling. Partition $\boldsymbol{X} = [\,\boldsymbol{X}_1 \;\; \boldsymbol{X}_2\,]$ with $\boldsymbol{X}_1 =$ `educ` and $\boldsymbol{X}_2 =$ everything else.

```
## Step 1: residualize the treatment on the controls
d_tilde <- residuals(lm(educ ~ exper + I(exper^2) + female + black, data = gss))

## Step 2: residualize the outcome on the controls
y_tilde <- residuals(lm(lnearn ~ exper + I(exper^2) + female + black, data = gss))

## Step 3: bivariate regression of the residuals
b_fwl <- coef(lm(y_tilde ~ d_tilde))["d_tilde"]

c(full_regression = coef(fit)["educ"], fwl = b_fwl)
#> full_regression.educ          fwl.d_tilde
#>               0.1405               0.1405
```

Both forms of the regression-anatomy formula:

```
c(cov_ytilde_dtilde = cov(y_tilde, d_tilde) / var(d_tilde),
  cov_y_dtilde      = cov(gss$lnearn, d_tilde) / var(d_tilde))
#> cov_ytilde_dtilde      cov_y_dtilde
#>            0.1405            0.1405
```

In fact *all three* numerators agree, for the same reason:

```
c(both_residualized = cov(y_tilde, d_tilde) / var(d_tilde),
  only_regressor    = cov(gss$lnearn, d_tilde) / var(d_tilde),
  only_outcome      = cov(y_tilde, gss$educ) / var(d_tilde))
#> both_residualized    only_regressor      only_outcome
#>            0.1405            0.1405            0.1405
```

What you *cannot* change is the **denominator**:

```
c(correct = coef(lm(y_tilde ~ d_tilde))["d_tilde"],
  wrong   = coef(lm(y_tilde ~ gss$educ))[2])
#> correct.d_tilde  wrong.gss$educ
#>          0.1405          0.1293
```

> **Note**
>
> **Why does the second one fail?** Regressing $\tilde{Y}$ on raw `educ` divides by $\mathrm{Var}_n(\texttt{educ})$ instead of $\mathrm{Var}_n(\widetilde{\texttt{educ}})$. Since residualizing can only shrink the variance, the denominator is too large and the estimate is attenuated. The numerators are all equal because the part removed from Y lies in the span of the controls and is orthogonal to $\tilde{D}$. Work this out on

paper; it is Question 2 of Problem Set 1.

The residuals are also identical, as the theorem promises:

```
all.equal(unname(residuals(lm(y_tilde ~ d_tilde))), unname(residuals(fit)))
#> [1] TRUE
```

7.1 The added-variable plot

```
tibble::tibble(d_tilde, y_tilde) |>
  ggplot(aes(d_tilde, y_tilde)) +
  geom_point(alpha = .12, size = .5, colour = "navy") +
  geom_smooth(method = "lm", se = FALSE, colour = "firebrick", linewidth = .7) +
  labs(x = "Years of schooling | controls", y = "Log earnings | controls") +
  theme_minimal(base_size = 10)
```

Figure 3: Added-variable plot for years of schooling. The slope of the fitted line *is* the multivariate coefficient on `educ`.

Note how much of the spread in `educ` survives residualization (`sd(d_tilde)` $= 2.71$ versus `sd(educ)` $= 2.82$). When that ratio gets small, the coefficient is being estimated off a thin slice of the data, and its standard error will show it.

8 Omitted variable bias, numerically

Short regression omits verbal ability and parental education; long regression includes them.

```
long  <- lm(lnearn ~ educ + exper + I(exper^2) + female + black +
              wordsum + pareduc, data = gss)
short <- lm(lnearn ~ educ + exper + I(exper^2) + female + black, data = gss)

## auxiliary regressions: each omitted variable on the included ones
aux_w <- lm(wordsum ~ educ + exper + I(exper^2) + female + black, data = gss)
aux_p <- lm(pareduc ~ educ + exper + I(exper^2) + female + black, data = gss)

beta_short <- coef(short)["educ"]
beta_long  <- coef(long)["educ"]
bias_pred  <- coef(long)["wordsum"] * coef(aux_w)["educ"] +
              coef(long)["pareduc"] * coef(aux_p)["educ"]

round(c(short = beta_short,
        long = beta_long,
        long_plus_bias = beta_long + bias_pred,
        bias = bias_pred), 5)
#>          short.educ           long.educ long_plus_bias.educ        bias.wordsum
#>              0.14053             0.11829             0.14053             0.02225
```

The identity $\tilde{\beta}_1 = \beta_1 + \sum_j \beta_{2j}\delta_{1j}$ holds exactly, to machine precision. Note the direction: ability and family background are positively associated with schooling and positively associated with earnings, so omitting them **overstates** the return to schooling.

```
modelsummary(list("(1) Bivariate" = lm(lnearn ~ educ, data = gss),
                  "(2) + demographics" = short,
                  "(3) + ability, background" = long),
             coef_map = c("educ" = "Years of schooling",
                          "wordsum" = "Vocabulary score",
                          "pareduc" = "Parental education"),
             gof_map = c("nobs", "r.squared"),
             stars = TRUE, output = "markdown")
```

	(1) Bivariate	(2) + demographics	(3) + ability, background
Years of schooling	0.119***	0.141***	0.118***
	(0.005)	(0.005)	(0.006)
Vocabulary score			0.041***
			(0.008)
Parental education			0.021***
			(0.004)
Num.Obs.	3509	3509	3509
R2	0.129	0.233	0.245

	(1) Bivariate	(2) • demographics	(3) • ability, background
• p < 0.1, * p < 0.05, ** p < 0.01, *** p < 0.001			

> **Important**
>
> Everything in this section is *algebra*. It tells you how the estimate moves when you add a control. It does **not** tell you whether the long regression recovers the causal return to schooling, for that you would need to argue that nothing else is omitted, which is a substantive claim, not a statistical one. That argument is what Weeks 5–11 are about.

9 Interpretation: functional form

9.1 Logs

With $\ln Y$ on the left, a coefficient β is approximately the proportional change in Y per unit of X. The exact percentage change is $100(e^{\beta} - 1)$.

```
b <- coef(long)["educ"]
c(approx_pct = 100 * b, exact_pct = 100 * (exp(b) - 1))
#> approx_pct.educ  exact_pct.educ
#>           11.83           12.56
```

For small β the two agree; by $\beta = 0.5$ they do not (50% vs 64.9%).

9.2 Quadratics

`exper` enters quadratically, so the marginal effect depends on where you evaluate it:

```
b1 <- coef(long)["exper"]; b2 <- coef(long)["I(exper^2)"]
sapply(c(0, 10, 20, 30, 40), function(x) b1 + 2 * b2 * x) |>
  setNames(paste0("exper=", c(0, 10, 20, 30, 40))) |> round(4)
#>  exper=0 exper=10 exper=20 exper=30 exper=40
#>   0.0541   0.0390   0.0238   0.0087  -0.0065

-b1 / (2 * b2)      # turning point, in years of experience
#> exper
#> 35.73
```

9.3 Interactions

```
fit_int <- lm(lnearn ~ educ * female + exper + I(exper^2) + black +
                wordsum + pareduc, data = gss)
round(coef(fit_int)[c("educ", "female", "educ:female")], 4)
```

```
#>        educ      female educ:female
#>      0.1055     -0.7629      0.0270
```

The return to schooling is `educ` for men and `educ + educ:female` for women. Never read the `female` main effect as "the gender gap": with an interaction it is the gap at `educ == 0`, which is outside the data.

9.4 Saturating the key covariate

```
fit_sat <- lm(lnearn ~ factor(educ) + exper + I(exper^2) + female + black +
                wordsum + pareduc, data = gss)
c(linear_R2 = summary(long)$r.squared,
  saturated_R2 = summary(fit_sat)$r.squared)
#>    linear_R2 saturated_R2
#>       0.2448       0.2633
anova(long, fit_sat)
#> Analysis of Variance Table
#>
#> Model 1: lnearn ~ educ + exper + I(exper^2) + female + black + wordsum +
#>     pareduc
#> Model 2: lnearn ~ factor(educ) + exper + I(exper^2) + female + black +
#>     wordsum + pareduc
#>   Res.Df  RSS Df Sum of Sq    F         Pr(>F)
#> 1   3501 2313
#> 2   3483 2256 18      56.7 4.86 0.000000000057 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The F-test here is exactly the model comparison from *Social Statistics I*: is the reduction in SSE from letting every year of schooling have its own coefficient large enough to justify the extra parameters?

10 Exercises

Work these during lab; they are not collected, but Problem Set 1 builds directly on them.

1. **Rank.** Construct a design matrix that includes a full set of nine `region` dummies *and* an intercept. Confirm with `qr()$rank` that it is rank deficient, and confirm what `lm()` does about it.

2. **The trace identity.** On the 300-observation subsample, verify numerically that $E[\hat{\boldsymbol{e}}'\hat{\boldsymbol{e}}] = \sigma^2(n-k)$ by simulation: fix `Xs`, draw $\boldsymbol{\epsilon} \sim N(0, 4\boldsymbol{I})$, generate $\boldsymbol{y} = \boldsymbol{X}_s\boldsymbol{\beta} + \boldsymbol{\epsilon}$ for some $\boldsymbol{\beta}$ you choose, and average $\hat{\boldsymbol{e}}'\hat{\boldsymbol{e}}$ over 2,000 replications. Compare to $4 \times (300 - k)$.

3. **FWL as fixed effects.** Regress `lnearn` on `educ` and a full set of `region` dummies. Then

(a) demean `lnearn` and `educ` within `region` and regress the demeaned outcome on the demeaned regressor. Confirm the slopes match. This is the within estimator we will meet in Week 10.

4. **Sign the bias.** Suppose you omit `hours` from the long regression. Predict the sign of the bias in the `educ` coefficient *before* running anything, using the OVB formula and the two auxiliary regressions. Then check.

5. **Leverage.** Find the ten highest-leverage observations. What makes them unusual? Refit dropping them, how much does the `educ` coefficient move? (Compare to its standard error before concluding anything.)

6. **Variance inflation.** Compute $\mathrm{Var}(\hat{\beta}_{\texttt{educ}})$ from the formula $s^2 / \sum_i \tilde{d}_i^2$ using the residualized `d_tilde` from the FWL section, and confirm it matches the `educ` entry of $s^2(\boldsymbol{X}'\boldsymbol{X})^{-1}$.

11 Session information

```
sessionInfo()
#> R version 4.4.1 (2024-06-14)
#> Platform: aarch64-apple-darwin20
#> Running under: macOS 26.5.2
#>
#> Matrix products: default
#> BLAS:   /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRblas.0.dylib
#> LAPACK: /Library/Frameworks/R.framework/Versions/4.4-arm64/Resources/lib/libRlapack.dylib;
#>
#> locale:
#> [1] C.UTF-8/C.UTF-8/C.UTF-8/C/C.UTF-8/C.UTF-8
#>
#> time zone: Asia/Shanghai
#> tzcode source: internal
#>
#> attached base packages:
#> [1] stats     graphics  grDevices utils     datasets  methods   base
#>
#> other attached packages:
#> [1] modelsummary_2.6.0 broom_1.0.7        ggplot2_4.0.0      dplyr_1.1.4
#>
#> loaded via a namespace (and not attached):
#>  [1] gtable_0.3.6        xfun_0.47           bayestestR_0.17.0
#>  [4] insight_1.4.6       lattice_0.22-7      vctrs_0.6.5
#>  [7] tools_4.4.1         generics_0.1.3      parallel_4.4.1
#> [10] datawizard_1.3.0    sandwich_3.1-1      tibble_3.2.1
#> [13] fansi_1.0.6         pkgconfig_2.0.3     Matrix_1.7-0
```

```
#> [16] tinytable_0.16.0    data.table_1.18.2.1 checkmate_2.3.2
#> [19] RColorBrewer_1.1-3  S7_0.2.0            lifecycle_1.0.4
#> [22] compiler_4.4.1      farver_2.1.2        tinytex_0.53
#> [25] codetools_0.2-20    htmltools_0.5.8.1   yaml_2.3.10
#> [28] pillar_1.9.0        tidyr_1.3.1         MASS_7.3-60.2
#> [31] multcomp_1.4-29     nlme_3.1-164        parallelly_1.45.0
#> [34] tidyselect_1.2.1    digest_0.6.37       performance_0.16.0
#> [37] mvtnorm_1.3-1       future_1.58.0       purrr_1.0.2
#> [40] listenv_0.9.1       labeling_0.4.3      splines_4.4.1
#> [43] fastmap_1.2.0       grid_4.4.1          cli_3.6.5
#> [46] magrittr_2.0.3      survival_3.6-4      utf8_1.2.4
#> [49] future.apply_1.20.0 TH.data_1.1-4       withr_3.0.1
#> [52] scales_1.4.0        backports_1.5.0     estimability_1.5.1
#> [55] rmarkdown_2.28      emmeans_1.11.1      globals_0.18.0
#> [58] zoo_1.8-12          coda_0.19-4.1       evaluate_1.0.0
#> [61] knitr_1.48          parameters_0.28.3   lmtest_0.9-40
#> [64] mgcv_1.9-1          rlang_1.3.0         xtable_1.8-4
#> [67] glue_1.7.0          jsonlite_1.8.9      R6_2.5.1
#> [70] tables_0.9.33
```